Reporte de Investigación: Conectividad entre Dispositivos — Crozzo POS

> Fecha: 2026-11-08 · Objetivo: analizar el estado real de la conectividad, opciones para emparejamiento sin fricción, y roadmap de mejora.

---

1. Estado real del código (lo que YA existe)

1.1 Servidor LAN en Rust (src-tauri/src/)

Componente	Archivo	Estado	Detalle
Servidor HTTP (puerto 3000)	`crozzo_lan_sync_server.rs`	✅ Implementado	Endpoints: /health, /status, /api/{sync,orders,sales,invoices,comandas}
WebSocket push (puerto 3001)	`crozzo_lan_ws.rs`	✅ Implementado	Broadcast en tiempo real a tablets conectadas
mDNS descubrimiento	`crozzo_mdns.rs`	✅ Implementado	Anuncia `_crozzo-pos._tcp.local.`, descubre peers automáticamente
Auth token (x-crozzo-lan-token)	`crozzo_lan_sync_server.rs`	✅ Implementado	Token de pareo, validación en cada request de escritura
Persistencia (SQLite)	`crozzo_lan_persistence`	✅ Implementado	Operaciones pendientes persistidas
Rate limiting (RUT)	`crozzo_lan_sync_server.rs`	✅ Implementado	1 consulta/6s por IP
CORS + semaphore (64 conn)	`crozzo_lan_sync_server.rs`	✅ Implementado	Configurable por env CROZZO_LAN_MAX_HTTP
Cloud mirror + drain	`CrozzoPosRuntimeCloud.js`	✅ Implementado	Supabase, outbox, ACK
Sync comandas LAN	`CrozzoLanOpsSync.js`	✅ Implementado	Pull/push WS + HTTP, digest

1.2 Flujo de pareo actual

```
1. Rol A (caja) inicia servidor LAN (mDNS anuncia _crozzo-pos._tcp)
2. Rol B (tablet) busca por mDNS → encuentra caja → obtiene IP:port + token
3. Tablet se autentica con token → comienza sync (HTTP poll + WS push)
4. Si hay internet → también mirror a Supabase (cloud)
```

1.3 Lo que YA funciona sin fricción
- **mDNS discovery**: si ambos dispositivos están en la misma red WiFi, la tablet encuentra la caja automáticamente. No se necesita IP manual.
- **Token de pareo**: se genera y comparte automáticamente vía QR (CrozzoPairingQrReader).
- **Cloud fallback**: si no hay LAN, sincroniza por Supabase.

---

2. Bugs latentes (problemas reales que hay que corregir)

#	Bug	Archivo	Causa	Impacto	Prioridad
1	**Digest LAN ciego**	`CrozzoLanOpsSync.js`	Hash solo IDs+count, no estado de comanda	Pull se omite cuando solo cambia estado → comanda "se pierde"	🔴 Alta
2	**WS broadcast panic**	`crozzo_lan_ws.rs`	`Handle::block_on` dentro de async context	Puede crashear el WS push → tablets dejan de recibir tiempo real	🔴 Alta
3	**Body 64KB single read**	`crozzo_lan_sync_server.rs`	Una sola lectura de 64KB	Truncación de payloads grandes (muchas comandas)	🟡 Media
4	**Outbox sin ACK 6h**	`CrozzoComandasCloudSync.js`	Sin idempotencia por consecutivo	Operaciones pueden duplicarse tras 6h sin ACK	🟡 Media
5	**Multi-writer carrito**	`CrozzoPosRuntimeCloud.js`	Sin lock fuerte, merge por timestamp	Carrito confundido con 2+ editores simultáneos	🟡 Media
6	**/api/comandas sin `since`**	`crozzo_lan_sync_server.rs`	Pull completo cada vez		

Ineficiente con muchas comandas | 🟢 Baja |

---

3. Opciones de conectividad: análisis de viabilidad

3.1 Lo que ya tenemos (LAN + Cloud)

Mecanismo	Disponibilidad	Fricción	Confiabilidad
mDNS (misma red WiFi)	✅ Implementado	Baja (automático)	Alta (misma subred)
HTTP polling (port 3000)	✅ Implementado	Ninguna	Alta
WebSocket push (port 3001)	✅ Implementado	Ninguna	Media (bug panic)
Cloud (Supabase)	✅ Implementado	Ninguna	Media (depende de internet)
QR pairing	✅ Implementado	Baja (escanear QR)	Alta

3.2 Opciones adicionales investigadas

A. WebRTC Data Channels (P2P directo)
- **Qué es**: Conexión peer-to-peer directa entre navegadores/dispositivos sin servidor central (solo signaling inicial).
- **Ventaja**: Latencia mínima, no satura base de datos, funciona en LAN e internet.
- **Desventaja**: Requiere signaling server (o Nostr relay) para establecer conexión inicial. NAT traversal puede fallar (requiere TURN relay).
- **Viabilidad en Tauri**: WebRTC está disponible en el webview (Chromium/WebView2). Se puede usar `RTCPeerConnection` desde JS.
- **Complejidad**: Media-Alta. Requiere implementar signaling + ICE + data channel.
- **Recomendación**: ✅ Viável como complemento, no como reemplazo. Útil para transferencia de archivos grandes (reportes, backups).

B. Bluetooth (Web Bluetooth API / Tauri plugin)
- **Qué es**: Comunicación directa por Bluetooth LE entre dispositivos cercanos.
- **Ventaja**: No requiere WiFi ni internet. Funciona en espacios sin red.
- **Desventaja**: Web Bluetooth solo funciona en Chrome/Edge (no Safari/Firefox). En Tauri, se necesita plugin nativo (`tauri-plugin-blec`). Alcance limitado (~10-30m). Ancho de banda bajo.
- **Viabilidad en Tauri**: Existe `tauri-plugin-blec` (cross-platform BLE client). Requiere permisos nativos.
- **Complejidad**: Alta. Requiere plugin Rust + permisos Android/iOS.
- **Recomendación**: ⚠️ Solo para casos específicos (impresoras Bluetooth, dispositivos sin WiFi). No para sync de datos.

C. WiFi Direct / Hotspot
- **Qué es**: Un dispositivo crea un hotspot WiFi, otros se conectan directamente.
- **Ventaja**: No requiere router ni internet.
- **Desventaja**: Requiere que un dispositivo actúe como AP. Complejidad de gestión de red. No soportado nativamente en web.
- **Viabilidad en Tauri**: Requiere APIs nativas de OS (Android/iOS/Windows). Tauri no tiene plugin estándar.
- **Complejidad**: Muy alta.
- **Recomendación**: ❌ No recomendado para POS. mDNS sobre WiFi existente es suficiente.

D. Transferencia por link (estilo "wormhole" / croc)
- **Qué es**: Un dispositivo genera un link/código; otro lo usa para descargar un archivo. Funciona por internet (relay) o LAN.
- **Ventaja**: Frictionless (solo compartir link). Funciona cross-network.
- **Desventaja**: Requiere relay server. No es tiempo real. Solo archivos, no sync bidireccional.
- **Viabilidad en Tauri**: Se puede implementar con un relay HTTP/WebSocket simple.
- **Complejidad**: Media.
- **Recomendación**: ✅ Útil para exportar reportes/backups a otro dispositivo sin saturar la DB.

E. Internet relay (señalización + TURN)

- **Qué es**: Servidor relay que facilita la conexión inicial entre dispositivos en redes diferentes.
- **Ventaja**: Funciona cross-network (internet).
- **Desventaja**: Requiere infraestructura. Latencia mayor. Costo de servidor.
- **Viabilidad**: Ya tenemos Supabase como relay para sync. Se puede extender.
- **Recomendación**: ✅ Ya implementado parcialmente (cloud sync). Mejorar con WebRTC para P2P directo cuando sea posible.

---

4. Permisos y fricción de usuario

4.1 Permisos necesarios por plataforma

Plataforma	Permisos necesarios	Cómo se piden
Android	`ACCESS_WIFI_STATE`, `CHANGE_WIFI_MULTICAST_STATE`, `BLUETOOTH`, `BLUETOOTH_ADMIN`, `ACCESS_FINE_LOCATION` (para mDNS/BLE)	`AndroidManifest.xml` + runtime request
iOS	`NSLocalNetworkUsageDescription`, `NSBonjourServices`, `NSBluetoothAlwaysUsageDescription`	`Info.plist` + runtime request
Windows (Tauri)	Ninguno especial (firewall puede bloquear port 3000)	Windows Firewall prompt en primer uso
macOS	Local network permission	System prompt en primer uso
Web (WebView)	`bluetooth` permission policy (solo Chrome)	`navigator.bluetooth.requestDevice()`

4.2 Flujo ideal de permisos (frictionless)

```
1. Usuario instala app → OS pide permisos de red/local (una vez)
2. App inicia → mDNS anuncia automáticamente
3. Tablet en misma red → descubre caja automáticamente (sin IP manual)
4. Si no encuentra → QR de pareo (escanea y listo)
5. Si no hay LAN → cloud sync (Supabase) automáticamente
6. Si no hay internet → modo offline (datos locales)
```

Clave: El usuario solo debe dar permisos UNA vez. Después, todo es automático.

---

5. Estrategia de conectividad multi-capa (recomendada)

```
CAPA 1 — mDNS (misma red, sin internet)
  └→ Automático si ambos en misma WiFi
  └→ Tablet descubre caja por _crozzo-pos._tcp
  └→ Token compartido vía QR (una vez)

CAPA 2 — Cloud (internet disponible)
  └→ Supabase como relay + persistencia
  └→ Funciona cross-network (diferentes redes)
  └→ Fallback automático si LAN falla

CAPA 3 — WebRTC P2P (LAN o internet, baja latencia)
  └→ Signaling vía Supabase (ya existe)
  └→ Data channel directo entre dispositivos
  └→ No satura la base de datos
  └→ Útil para transferencia de archivos grandes

CAPA 4 — Offline (sin red)
```

```
  └→ Datos locales (SQLite/localStorage)
  └→ Sync cuando vuelve conexión
  └→ Contingencia fiscal (48h SLA)
```

Prioridad de conexión (automática)
```
1. ¿Misma red LAN? → mDNS + HTTP/WS (más rápido)
2. ¿Hay internet? → Cloud sync (Supabase)
3. ¿LAN + internet? → Ambos en paralelo (redundancia)
4. ¿Sin red? → Offline (datos locales)
```

---

6. Lo que hay que corregir (bugs latentes)

#	Bug	Archivo	Fix propuesto	Prioridad
1	Digest LAN ciego	`CrozzoLanOpsSync.js`	Incluir estado en hash (no solo IDs+count)	🔴 Alta
2	WS broadcast panic	`crozzo_lan_ws.rs`	Reemplazar `Handle::block_on` por async await	🔴 Alta
3	Body 64KB truncation	`crozzo_lan_sync_server.rs`	Leer en loop hasta EOF o usar chunked	🟡 Media
4	Outbox sin idempotencia	`CrozzoComandasCloudSync.js`	Agregar consecutivo/idempotency key	🟡 Media
5	Multi-writer carrito	`CrozzoPosRuntimeCloud.js`	Lock optimista con version field	🟡 Media

---

7. Roadmap de mejora de conectividad

Fase A — Corregir bugs latentes (1-2 días)
- Fix digest LAN (incluir estado)
- Fix WS panic (async await)
- Fix body truncation (read loop)
- Agregar `since` param a /api/comandas

Fase B — Frictionless pairing (3-5 días)
- Verificar permisos en Android/iOS (manifest + runtime)
- Auto-discovery por mDNS (ya implementado, verificar en campo)
- QR de pareo como fallback (ya implementado)
- Cloud fallback automático (ya implementado)

Fase C — WebRTC P2P (1-2 semanas)
- Implementar signaling vía Supabase (ya existe infra)
- Data channel para sync de datos (no solo archivos)
- Fallback a HTTP/WS si WebRTC falla
- Transferencia de archivos grandes (reportes, backups)

Fase D — Transferencia por link (1 semana)
- Endpoint para generar link de descarga
- Descarga por código/link (estilo wormhole)
- Útil para exportar reportes sin saturar DB

---

8. Riesgos y limitaciones

Riesgo	Mitigación
mDNS bloqueado por firewall/router	Fallback a cloud sync
WebRTC NAT traversal falla	TURN relay (costo de servidor)
Bluetooth alcance limitado	Solo para impresoras/dispositivos cercanos
Permisos denegados por usuario	Guía de configuración en app
Red intermitente	Modo offline + sync al volver
Saturación de base de datos	WebRTC P2P para datos grandes

---

9. Conclusión

El sistema **ya tiene conectividad LAN real e implementada** (HTTP + WS + mDNS + auth + persistencia + cloud). No es "pendiente" ni "inexistente". Los problemas son bugs latentes específicos que se pueden corregir.

Para lograr el objetivo de "sin fricción":
1. **Corregir los 2 bugs críticos** (digest ciego + WS panic) → 1-2 días
2. **Verificar permisos** en Android/iOS → 1 día
3. **Implementar WebRTC P2P** como complemento → 1-2 semanas
4. **Transferencia por link** para archivos grandes → 1 semana

El resultado final: un celular se conecta a la caja automáticamente por mDNS (misma red) o por cloud (internet), sin configuración manual, con fallback a offline si no hay red.